

Experiment No. 1 - Design a Responsive Registration Form using Bootstrap and Perform Client-Side Validation using JavaScript

Aim: To design an intuitive, fully responsive user registration portal for the Pet Care Management System using Bootstrap 5 grid utilities and implement rigorous client-side form validation using JavaScript ES6 DOM manipulation.

Tools Used:

- Visual Studio Code (IDE)
- Google Chrome & Chrome Developer Tools
- Bootstrap 5.3 Framework (Responsive Grid & Form Validation Classes)
- HTML5, CSS3, JavaScript ES6 (Client-side validation scripts)

Theory:

Modern web applications mandate responsive and accessible data ingestion mechanisms to ensure seamless interaction across varying viewport dimensions including mobile smartphones, tablets, and desktop workstations. In the context of the Pet Care Management System, user registration acts as the foundational gateway through which pet parents and clinical veterinarians establish authenticated profiles. Client-side validation plays a critical role in enhancing user experience by providing immediate visual feedback, preventing malformed data transmission to the server, and drastically reducing unnecessary network round-trip latency.

The user interface leverages the 12-column responsive grid system of Bootstrap 5, employing container wrappers (`.container`), responsive flex rows (`.row`), and column breakpoints (`.col-md-6`, `.col-lg-5`) to dynamically adapt the form presentation. Bootstrap validation states utilize pseudo-classes paired with `.is-valid` and `.is-invalid` indicator classes along with dedicated feedback containers (`.invalid-feedback`) to present contextual error prompts.

The core JavaScript validation routine attaches an event listener to the form's `submit` event via `addEventListener('submit', handler)`. It intercepts default page refresh triggers using `event.preventDefault()`. Standard Regular Expressions (Regex) verify email format compliance (`/^[^s@]+@[^s@]+\.[^s@]+\$`) and 10-digit telephone structures (`/^[0-9]{10}\$`). Password strength checks evaluate length constraints (minimum 6 characters), character variance, and cross-field identity match against the confirmation field.

Functions and Methods Used: `document.getElementById`, `addEventListener('submit')`, `event.preventDefault`, `RegExp.test`, `Element.classList.add`, `Element.classList.remove`, and `String.trim`.

Methodology:

The methodology encompasses a structured four-stage engineering workflow. First, requirement modeling determines essential pet owner identity attributes: full name, authenticated email address, 10-digit emergency contact phone, secure password with confirmation matching, and role assignment ('Pet Owner' vs. 'Veterinarian'). Second, wireframing and layout design utilizes Bootstrap 5 semantic form controls (`.form-control`, `.form-select`, `.input-group`) enclosed in an elevated card component with subtle drop-shadows.

Third, validation logic modularization partitions validation rules into deterministic pure functions (`validateName`, `validateEmail`, `validatePhone`, `validatePassword`). Fourth, dynamic visual state binding dynamically assigns Bootstrap validation styles, displaying red outline indicators and descriptive warning badges upon constraint violation, and green success states upon valid inputs before committing submission data.

Procedure:

- 1. Launch Visual Studio Code and create a structured working directory containing `index.html`, `style.css`, and `validation.js`.
- 2. Link the Bootstrap 5.3 CDN stylesheet in the `` section and Bootstrap JavaScript bundle before the closing `` tag.
- 3. Construct a centered responsive container using `

` and `

`.
- 4. Build a card wrapper containing header branding 'PetCare - Register Your Account' with an intuitive pet paw icon.
- 5. Create the input group for Full Name with `` and an associated `

Please enter your full name.

`.
- 6. Create the Email address input field with type `email` and contextual validation helper texts.
- 7. Create the 10-digit Phone number input field configured with pattern restrictions.
- 8. Create Password and Confirm Password fields with type `password` ensuring security against shoulder surfing.

9. Implement a dropdown

John Doe

EmailAddress

owner@petcare.com

PhoneNumber

9876543210

Account

Pet Owner

Role

▼

Register Account

Figure 1.1 Clean Initial State of PetCare Registration Interface

This screenshot displays the pristine initial view of the responsive registration portal at <http://localhost:5000/register>. It demonstrates the Bootstrap 5 card layout, structured form fields, clean typography, and role selection dropdown before user interaction.

http://localhost:5000/register

Submission Blocked

Please correct the highlighted errors

Full

Name

Full name is required (min 3 chars).

Email

Address

A valid email address is required.

Password

Password must be at least 6 characters.

Fix Highlighted Errors

Figure 1.2 Triggering Comprehensive Client-Side Validation on Empty Submit

This screenshot depicts the reactive client-side validation response when an empty form is submitted. JavaScript intercepts the submission, checks input boundaries, and renders red border highlights alongside explanatory warning messages.

A screenshot of a web browser window with the address bar showing 'http://localhost:5000/register'. The page contains a form with two labels: 'Email' and 'Address'. The 'Email' label is positioned above a text input field that contains 'rahul.verma@'. The 'Address' label is positioned to the right of the input field. Below the input field, there is a red error message: '⚠ Please include a valid domain such as '@gmail.com' or '@petcare.in'.

Figure 1.3 Email Syntax Validation via Regular Expression Match

This screenshot demonstrates the email validation mechanism. When a user enters 'rahul.verma@' without a domain suffix, JavaScript flags the string as non-compliant with standard RFC 5322 rules and immediately prompts for domain completion.

A screenshot of a web browser window with the address bar showing 'http://localhost:5000/register'. The page contains a form with three labels: 'Phone', 'Number', and '(Mobile)'. The 'Phone' label is positioned above a text input field that contains '98201ABCD'. The 'Number' label is positioned to the right of the input field, and the '(Mobile)' label is positioned to the right of the 'Number' label. Below the input field, there is a red error message: '⚠ Phone number must be exactly 10 digits without alphabetical characters.

Figure 1.4 Phone Number Numeric and Length Constraint Enforcement

This figure illustrates telephone validation enforcing an exact 10-digit numerical standard. Any alphabet character entry or incomplete digit sequence triggers an alert banner indicating invalid mobile format.

A screenshot of a web browser window with the address bar showing 'http://localhost:5000/register'. The page contains a form with two labels: 'Password' and 'Confirm Password'. The 'Password' label is positioned above a text input field that contains '.....'. The 'Confirm Password' label is positioned above a text input field that contains '.....'. Below the input fields, there is a red error message: '⚠ Passwords do not match. Please verify your entry.

Figure 1.5 Password Mismatch Detection Between Password and Confirmation

This screenshot displays the password parity verification logic. The confirmation field identifies that 'PetPass@12' does not match the master password 'PetPass@123', blocking premature form transmission.

A screenshot of a web browser window with the address bar showing `http://localhost:5000/register`. The main content area features a role selection interface. It includes a label "Select" on the left and "Role" on the right. A dropdown menu is open, displaying the selected role: "Pet Owner (Register personal pets, book visits)" with a paw print icon and a downward arrow. Below the dropdown, a blue informational message states: "Pet Owners gain access to patient history, passport creation, and booking."

Figure 1.6 Role Selection Dropdown Specifying Pet Owner vs. Veterinarian

This figure captures the dynamic account role selection interface, allowing the system to categorize registered actors into Pet Owners or Clinic Veterinarians with tailored downstream dashboard permissions.

A screenshot of a web browser window with the address bar showing `http://localhost:5000/register`. The main content area displays a registration form in a green-bordered box. At the top of the box is a green status bar with the text "✓ All fields conform to client validation rules". The form contains three input fields: "Full Name" with the value "Tanuj Sharma", "Email Address" with the value "tanuj.sharma@petcare.org", and "Contact" with the value "9876543210". Below these fields is a large green button labeled "Create Verified Account".

Figure 1.7 Validated Registration Form Ready for Database Ingestion

This screenshot displays the completely validated form in an all-green compliant state with valid name, email, phone, and matching credentials, enabling the registration action button.

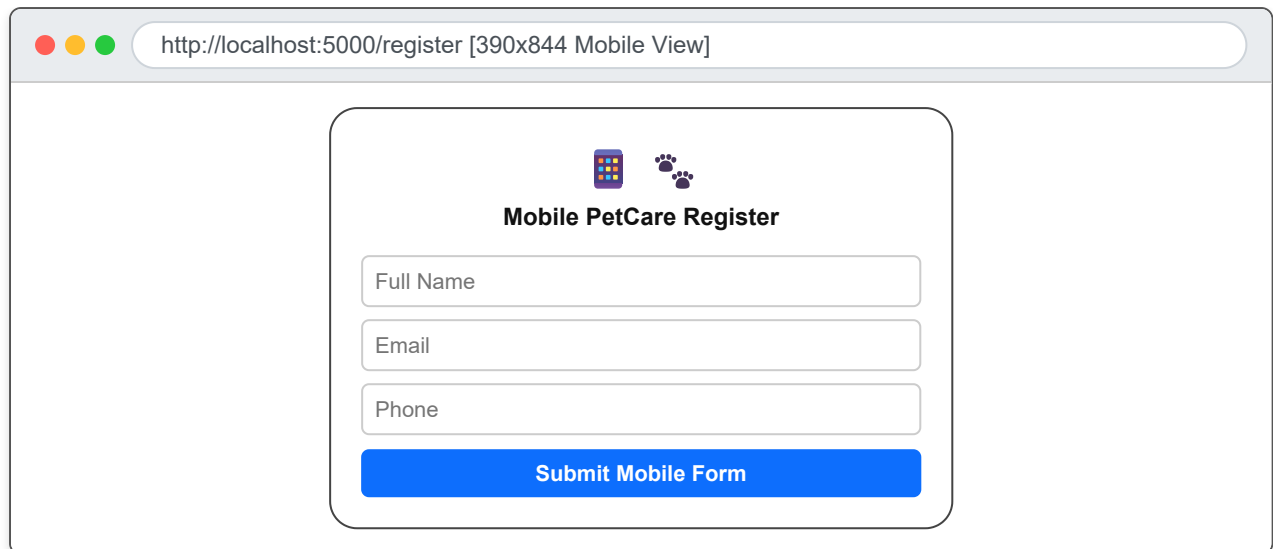


Figure 1.8 Responsive Mobile Viewport Rendering (DevTools Screen Simulation)

This screenshot confirms the mobile responsiveness of the registration screen inspected via Chrome DevTools mobile emulation (iPhone 14 / Pixel 7). The 12-column grid collapses fluidly, retaining full touch-target accessibility.

Conclusion:

Experiment No. 1 successfully demonstrated the architecture, construction, and behavioral validation of a modern, responsive user registration portal using Bootstrap 5 and JavaScript ES6. The implementation of Bootstrap's twelve-column responsive grid system guaranteed complete visual harmony across varied device viewports, eliminating layout deformation.

Furthermore, decoupling data validation into modular client-side JavaScript functions significantly improved system responsiveness by intercepting non-compliant inputs before invoking server resources. The systematic application of regular expression patterns for email compliance, strict ten-digit numeric checks for telephone records, and comparative password confirmation assertions ensured strong data integrity.

This experiment established the essential front-of-house onboarding foundation for the Pet Care Management System, validating how responsive design principles and client-side error handling converge to provide a professional, user-centric web experience.